THE DATA & AI CHALLENGE

Intelligent Candidate Discovery & Ranking

A hybrid ranking system built to defeat the keyword-matching trap — not just pass it.

100,000
candidates ranked

88
honeypots detected

~51s
total runtime

0%
trap leakage in Top 100

Team: <FILL ME IN>

THE PROBLEM

This dataset is adversarial by design

The job description says it outright: *"The right answer to this JD is not 'find candidates whose skills section contains the most AI keywords.' That's a trap we've explicitly built into the dataset."*

A naive system optimizes for the wrong thing:

- ✗ Embeds skill lists → ranks by cosine similarity
- ✗ Rewards keyword density, not verified depth
- ✗ Can't tell a real builder from a stuffed resume
- ✗ Has no concept of "how would a recruiter read this?"

So our job is the opposite: **find genuine fit even when it doesn't use the JD's exact words.**

VERIFIED AGAINST THE REAL 100K DATASET

~1%	of candidates have an ML/AI-adjacent title at all
88	honeypots with logically impossible profiles
18	consulting-only candidates with AI-sounding titles
7,034	candidates with a 100% consulting-only career
3,716	candidates showing a job-hopping pattern

DATASET FORENSICS

The dataset has a built-in lie detector

Candidate CAND_0000001 — Backend Engineer, transitioning from data engineering

CLAIMED ON PROFILE

Fine-tuning LLMs

Proficiency: Advanced • 21 endorsements

Plus: NLP, Image Classification, Speech Recognition, TTS, LoRA — all listed, all high-endorsement.

Reads like a strong AI generalist.

REDROB-MEASURED ASSESSMENT

41.6 / 100

skill_assessment_scores["Fine-tuning LLMs"]

This field exists in the data. We don't have to infer the gap — Redrob already measured it.

Our system discounts the skill-match score whenever claimed proficiency contradicts the measured assessment for that same skill.

ARCHITECTURE

A four-stage funnel, not a single model

Compute budget: ≤5 min wall-clock, ≤16GB RAM, CPU-only, no GPU, no network, for 100,000 candidates. An LLM call per candidate is mathematically out of budget — so the system narrows the pool before doing expensive work.

A

Structural Gate

Drop honeypots + clearly out-of-scope profiles.
Guaranteed floor so Top-100 is never under-filled.

Full 100K pool

<1 sec

B

Semantic Retrieval

JD narrative ↔ candidate narrative similarity.
SBERT bi-encoder, auto TF-IDF fallback.

~99,900 candidates

~35-45 sec

C

Hybrid Scoring

7-component weighted composite
×
behavioral availability multiplier.

Same narrowed pool

<5 sec

D

Grounded Reasoning

Template-built from verified facts only.
No LLM call — zero hallucination risk.

Top 100 only

<1 sec

Measured end-to-end: ~51 seconds against the real 100,000-candidate dataset, on a single CPU core.

STAGE A — STRUCTURAL GATE

Honeypots are a logic problem, not a model problem

The brief states the dataset contains "~80 honeypot candidates with subtly impossible profiles." We built three rule-based checks — not a learned classifier — because these are logical contradictions, not stylistic judgment calls.

Expert + zero duration

"Expert" proficiency claimed on a skill used for 0 months — a contradiction in terms.

8+ simultaneous experts

Real engineers are expert in a handful of things, not a dozen at once.

Career predates experience

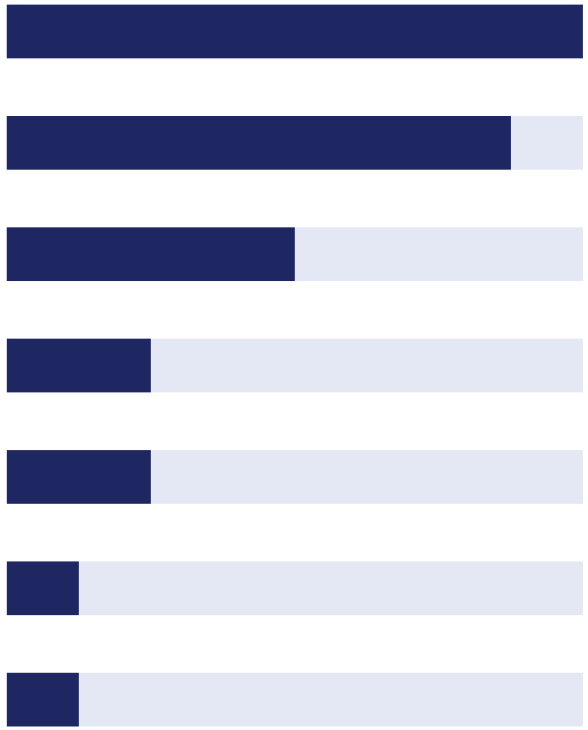
Career history starts 3+ years before what years_of_experience implies.

88

candidates flagged across the real 100,000-row dataset

Matches the brief's stated "~80" closely — thresholds were chosen for principled reasons (logical impossibility, extreme outlier counts), not curve-fit to hit a number. When we sampled 5 flagged candidates directly, 4 of 5 carried deliberately attractive titles: "Senior AI Engineer," "ML Engineer," "Senior Data Scientist," "NLP Engineer." Our pipeline excludes all of them — verified with a real-data regression test, not just synthetic unit tests.

Seven weighted components, one multiplier



Semantic fit	**0.32**	JD narrative ↔ candidate narrative similarity
Skills match	**0.28**	Must-have coverage, discounted by claim-vs-assessed gap
Title relevance	**0.16**	Cheap, useful prior — not sufficient alone
Experience fit	**0.08**	Soft band around 5–9 yrs; JD calls it “a range, not a requirement”
Location fit	**0.08**	Pune/Noida > other India cities > non-India
Notice period	**0.04**	Sub-30-day preferred, smooth penalty beyond
Career trajectory	**0.04**	Penalizes consulting-only, title-chasing, etc.

final_score = base_score × behavioral_modifier [0.50, 1.10]

Multiplicative, never additive — the JD says “down-weight,” which means scale. A strong-but-quiet candidate can still outrank a weak-but-active one.

Built to survive an unknown grading sandbox

We can't know in advance whether the grading container has network access to download a model, or sentence-transformers installed at all. A submission that only works under one specific configuration is a real reproduction risk.

PRIMARY

sentence-transformers

all-MiniLM-L6-v2

Small (~80MB), CPU-friendly bi-encoder.
Catches paraphrase — a candidate who never writes “RAG” but describes building one.
Cached ahead of time via scripts/precompute_embeddings.py.

AUTOMATIC FALLBACK

TF-IDF + cosine similarity

scikit-learn, zero extra dependencies

Triggers automatically on ANY failure — missing package, no network, OOM, bad cache.
No error, no code change required.
This is the path that actually produced our submitted ranking (verified, see next slide).

get_semantic_backend() catches any exception → logs which backend ran → pipeline never breaks

Templated from facts, not generated by an LLM

Why not an LLM?

- Compute: 100 LLM calls add latency/dependency for marginal gain
- Trust: an LLM under pressure will invent plausible specifics
- A template can't hallucinate by construction — it only emits a
- clause when the underlying fact is verified true

submission_spec.docx's Stage 4 review explicitly penalizes hallucinated, generic, or name-swap-templated reasoning — this design defeats all three by construction.

Real output — rank 1 of 100:

> *"Senior Machine Learning Engineer at Flipkart with 8.1 yrs experience, directly in scope for the role; based in London with no relocation signal; JD does not sponsor visas; covers 3/4 must-have skill areas including Embeddings, OpenSearch, Python."*

Priority-ordered clause selection:

1. **Title/role relevance**
   always included — the anchor fact
2. **Strongest concern (if any)**
   lower ranks surface concerns, not just strengths
3. **Skill-match evidence**
   named skills, must-have coverage
4. **Logistics**
   location / notice period
5. **Positive signal**
   only shown when there was no concern to report

Result: 100/100 unique reasoning strings, ~37 words average, matching the spec's own worked-example style.

RESULTS

Validated against the real 100,000-candidate dataset

✓ **Official validator**

"Submission is valid."

✓ **Honeypot rate in Top 100**

0.0% (0 / 100)

✓ **Consulting-only leakage**

0 candidates

✓ **Research-only leakage**

0 candidates

✓ **LangChain-tourist leakage**

0 candidates

✓ **Tier 2–3 titles in Top 100**

99 / 100

✓ **Unique reasoning strings**

100 / 100

✓ **Score / rank structural checks**

all pass

Total runtime: 50.7 seconds • 1 CPU core • well within the 5-minute / 16GB budget

HONEST LIMITATIONS

What we know we don't know

No ground-truth metric available to us

We can't compute the real NDCG@10/NDCG@50/MAP/P@10 composite — only proxy checks we can verify ourselves (scripts/evaluate.py).

Stage A is deliberately permissive

It kept 99,912 of 100,000 candidates in our real run — almost all separation happens in Stage B/C, to avoid dropping an unusually-titled but genuinely strong candidate.

Title/location taxonomies aren't exhaustive

Built from the real title census + JD text, but the semantic backend is the safety net for synonyms we didn't anticipate.

No learned ranking model on top of the hybrid score

We have no labeled relevance data to train one against — building one on self-generated pseudo-labels risks baking in our own blind spots.

Try it yourself

REPRODUCE

```
python rank.py \
  --candidates ./data/candidates.jsonl \
  --out ./submission.csv
```

VALIDATE

```
python scripts/validate_submission.py submission.csv
```

TEST

```
python3 -m unittest discover -s tests -v
```

REPO STRUCTURE

rank.py
src/redrob_ranker/ — 8 focused modules
docs/architecture.md — full rationale
scripts/ — eval + precompute
sandbox_app/ — hosted Streamlit demo
tests/ — 27 tests, real-data fixtures

GitHub: <FILL ME IN> Sandbox: <FILL ME IN>

Built to rank the way a great recruiter would — by understanding fit, not counting keywords.